

Incident handler's journal

Incident response journal entries.

Date: 19 July 2026	Entry: 1
Description	Analysis of a ransomware incident at a small U.S. health care clinic. The clinic's files were encrypted after an employee downloaded a malicious attachment from a phishing email, halting business operations and blocking access to patient records. This entry documents the incident using the 5 W's.
Tool(s) used	None at this stage. This entry is a documentation and analysis exercise based on the reported scenario rather than a hands-on investigation. Tools that would support a real response include a SIEM for log analysis, endpoint detection and response for the affected workstations, and email security filtering to trace the phishing source.
The 5 W's	• Who: An organized ransomware group known to target the healthcare and transportation sectors. The attackers gained entry through targeted phishing emails sent to several employees. • What: A phishing email carried a malicious attachment that, once downloaded, installed malware on an employee's computer. The attackers deployed ransomware, which encrypted critical files. A ransom note demanded payment for the decryption key. • When: Tuesday at approximately 9:00 a.m., when employees found they could not access files such as medical records. • Where: On the network of a small U.S. health care clinic providing primary-care services. The compromise began on an employee workstation and spread to file systems holding patient data. • Why: The attackers were financially motivated and sought a ransom payment. The entry point was human: employees were tricked into downloading a malicious attachment.
Additional notes	The root cause was a human-targeted phishing email, so security awareness training and stronger email filtering are the first preventive priorities. Offline, tested backups would reduce the leverage of a ransom demand. Open questions: were files exfiltrated before encryption, which could trigger a HIPAA breach assessment and potential notification obligations, and was network segmentation in place to limit spread?

Date: 21 July 2026	Entry: 2
------------------------------	--------------------

Description	Investigation and resolution of a phishing alert (ticket A-2703) at a financial services company, following the organization's phishing incident response playbook. The alert concerned a suspicious email attachment whose file hash had previously been verified as malicious.
Tool(s) used	Phishing incident response playbook and flowchart, and the alert ticketing system. Threat intelligence such as VirusTotal for file-hash reputation, referenced from the earlier verification of the attachment hash.
The 5 W's	• Who: An external threat actor posing as a job applicant, "Clyde West" of "Def Communications," using a spoofed sender address to target the company's HR team. • What: A phishing email delivered a password-protected executable (<code>bfsvc.exe</code>) disguised as a resume and cover letter. The file hash is confirmed malicious, and the recipient may have downloaded and opened the attachment. • When: The email was sent on Wednesday, 20 July 2022 at 09:30 a.m., and ticket A-2703 was raised for investigation. • Where: At a financial services company, targeting the HR mailbox (<code>hr@inergy.com</code>) through the mail server (SERVER-MAIL). The affected endpoint is the recipient employee's computer. • Why: To trick an HR employee, who routinely opens resumes, into executing malware and gaining a foothold on the company network, likely for financial gain given the target sector.
Additional notes	The password-protected <code>.exe</code> is a deliberate evasion technique to bypass email scanning, and no legitimate applicant sends a password-protected executable as a resume. Because the attachment is confirmed malicious, the playbook directs escalation to a level-two analyst rather than closure. Follow-up for tier two: did the endpoint execute the file, and does EDR show signs of compromise or lateral movement? The affected host should be isolated pending that review.

Date: 20 July 2026	Entry: 3
Description	Analyzed a sample packet capture (<code>sample.pcap</code>) in Wireshark as a security analyst investigating traffic to a website, learning to read packet data and apply display filters to isolate the traffic relevant to an investigation. This entry documents the use of a cybersecurity tool.

Tool(s) used	Wireshark. I opened <code>sample.pcap</code> and read the packet-list columns (No., Time, Source, Destination, Protocol, Length, Info), using Wireshark's coloring rules to classify traffic types at a glance. I then applied display filters to narrow the capture: by IP address (<code>ip.addr</code>, <code>ip.src</code>, and <code>ip.dst == 142.250.1.139</code>), by Ethernet MAC address (<code>eth.addr == 42:01:ac:15:e0:02</code>), and by protocol and port (<code>udp.port == 53</code> for DNS, <code>tcp.port == 80</code> for HTTP, and <code>tcp contains "curl"</code> to search payload text). To inspect a single packet I opened the details pane and expanded its protocol stack (the Frame, Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol subtrees) to read fields such as TTL, header length, and TCP flags.
Findings from the capture	• The first packet whose Info column showed <code>Echo (ping) request</code> used the ICMP protocol. • A TCP packet's destination port was 80, the default HTTP port. • The DNS traffic (<code>udp.port == 53</code>) resolved the query for opensource.google.com to 142.250.1.139 (shown in the Answers section). • A separate HTTP packet in the same capture, a <code>curl</code> request examined under the <code>tcp.port == 80</code> filter, had its own destination of 169.254.169.254, a TTL of 64, a frame length of 54 bytes, and an IP header length of 20 bytes. It is a different trace from the DNS lookup above, so its destination is not the resolved 142.250.1.139.
Additional notes	The core skill is using layered display filters (by address, MAC, protocol, port, and payload content) to reduce a large capture to just the packets that matter, then reading one packet's protocol stack from the frame up through TCP. <i>Completed in the course Linux and Windows lab environment. The capture file (<code>sample.pcap</code>) remained on the lab machine, so this entry records the workflow and findings rather than linking a saved artifact.</i>
Date: 20 July 2026	Entry: 4
Description	Used tcpdump on a Linux virtual machine (as the user <code>analyst</code>) to identify network interfaces, capture live traffic to a packet-capture file, and filter the saved capture, the command-line counterpart to the Wireshark analysis in Entry 3. This entry documents the use of a cybersecurity tool.

Tool(s) used	tcpdump. I identified available capture interfaces with <code>sudo ifconfig</code> and <code>sudo tcpdump -D</code>, selecting <code>eth0</code>, then inspected live traffic with <code>sudo tcpdump -i eth0 -v -c5</code> (<code>-i</code> interface, <code>-v</code> verbose, <code>-c5</code> five packets), reading the link type, timestamps, IP fields (TOS, TTL, offset, flags, protocol, length), the source-to-destination flow with ports, and the TCP flags, checksum, and sequence and acknowledgment numbers. I captured to a file with <code>sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &</code> (<code>-nn</code> no name resolution, <code>port 80</code> HTTP only, <code>-c9</code> nine packets, <code>-w</code> write to file, <code>&</code> run in the background), generated traffic with <code>curl opensource.google.com</code>, and verified the file with <code>ls -l capture.pcap</code>. Finally I read the saved capture back with <code>sudo tcpdump -nn -r capture.pcap -v</code> (<code>-r</code> read from file) and inspected packet payloads in hexadecimal and ASCII with <code>-X</code>.
Additional notes	Two habits stood out. First, disabling name resolution with <code>-nn</code> during an investigation, because a reverse lookup can alert a malicious actor and may return unreliable data. Second, scoping a capture tightly (a single interface, one port, a fixed packet count, written to a file) so the evidence stays small, relevant, and reviewable later. Together with Entry 3, this covers both halves of packet work: capturing on the command line and analyzing in a GUI. <i>Completed in the course Linux lab environment. The capture file remained on the lab machine, so this entry records the commands, options, and workflow rather than linking a saved artifact.</i>

Date: 20 July 2026	Entry: 5
Description	Threat-intelligence analysis of the malicious attachment from the phishing alert (ticket A-2703), investigating its file hash in VirusTotal to extract indicators of compromise and map them onto the Pyramid of Pain. This is the tool-driven follow-on to the playbook triage in Entry 2, and it documents the use of a cybersecurity tool.
Tool(s) used	VirusTotal. I searched the attachment's file hash and reviewed four tabs: Detection (vendor flags), Details (static properties and additional hashes), Relations (contacted domains, URLs, IPs), and Behavior (sandbox activity mapped to MITRE ATT&CK techniques). Maliciousness was confirmed from several agreeing signals rather than a single number: 51 of 69 vendors flagged the file, the community score was -297, and the threat label <code>trojan.flagpro/fragtor</code> identified it as the Flagpro malware family. The file masqueraded as a legitimate Windows binary (<code>bfsvc.exe</code>, the Boot File Servicing Utility) but was unsigned, and the sandbox observed process injection, sandbox evasion, and command-and-control traffic.

Indicators of compromise (Pyramid of Pain)	<i>This entry documents tool use and threat intelligence. The incident's 5 W's are recorded in Entry 2. I selected three indicators of compromise, one each from the base, middle, and top of the pyramid so the framework is visible in the result.</i> • Hash values (bottom, easiest to change): additional MD5 and SHA-1 hashes for the same file. Blocking a hash stops this exact file but not a recompiled copy. • Domain names (middle): <code>org.misecure.com</code>, a contacted domain matching an IDS rule for dynamic-DNS command-and-control traffic. Blocking it forces the attacker to stand up new infrastructure. • Tactics, techniques, and procedures (top, hardest to change): process injection (T1055), masquerading (T1036), virtualization and sandbox evasion (T1497), and application-layer C2 (T1071). Detecting on behaviour is the most durable defence.
Additional notes	The higher up the Pyramid of Pain a defender detects, the more costly the attack becomes to sustain, so behaviour-based detection (the TTPs) is the most durable outcome of this analysis. Recommended tier-two follow-up: confirm whether the endpoint executed the file, review EDR for compromise or lateral movement, and isolate the affected host pending that review.

Date: 22 July 2026	Entry: 6
Description	Explored intrusion detection with Suricata as a security analyst monitoring network traffic: examined the anatomy of a custom rule (signature), ran Suricata against a sample capture to trigger alerts, and analyzed the resulting <code>fast.log</code> and <code>eve.json</code> logs. This entry documents the use of a cybersecurity tool.
Tool(s) used	Suricata (with <code>jq</code> for JSON log analysis). I examined a custom rule in <code>custom.rules</code> with <code>cat</code>, breaking the signature into its three parts: the action (<code>alert</code>, alongside <code>drop</code>, <code>pass</code>, and <code>reject</code>), the header (protocol, source and destination networks and ports, and direction, for example <code>http \$HOME_NET any -> \$EXTERNAL_NET any</code>, where <code>\$HOME_NET</code> is the <code>172.21.224.0/20</code> variable defined in <code>/etc/suricata/suricata.yaml</code>), and the rule options (<code>msg:"GET on wire"</code>, <code>flow:established,to_server</code>, <code>content:"GET"</code>, <code>http_method</code>, <code>sid:12345</code>, <code>rev:3</code>). I ran the rule against the capture with <code>sudo suricata -r sample.pcap -S custom.rules -k none</code> (<code>-r</code> read the pcap, <code>-S</code> use the custom rule file, <code>-k none</code> disable checksum checks), which wrote alerts to <code>/var/log/suricata/fast.log</code> and detailed events to <code>eve.json</code>. I read <code>fast.log</code> with <code>cat</code> and analyzed <code>eve.json</code> with <code>jq</code>: pretty-printing with <code>jq . less</code>, extracting fields with <code>jq -c "[.timestamp,.flow_id,.alert.signature,.proto,.dest_ip]"</code>, and correlating a single network flow with <code>jq "select(.flow_id==...)"</code>.

Findings from the run	- The rule triggers whenever Suricata observes <code>GET</code> as the HTTP method in an HTTP packet leaving the home network for the external network (alert message "GET on wire"). - The first alert had a severity of 3, and its signature was "GET on wire". <code>fast.log</code> is a quick, line-based alert format useful for QA checks, <code>eve.json</code> is the standard, detailed JSON event log, and the <code>flow_id</code> field ties together all packets belonging to the same network flow.
Additional notes	The action, header, and options structure is the reusable mental model for reading or writing any IDS or IPS signature, and the <code>alert</code> versus <code>drop</code> distinction is the line between detection (IDS) and prevention (IPS mode, where <code>drop</code> actually takes effect). <code>jq</code> turns Suricata's verbose <code>eve.json</code> into targeted, greppable evidence by selecting specific fields and pivoting on <code>flow_id</code> to reconstruct a single conversation. <i>Completed in the course Linux lab environment. <code>sample.pcap</code> and <code>custom.rules</code> remained on the lab machine, so this entry records the workflow, commands, and findings rather than linking saved artifacts.</i>

Date: 22 July 2026	Entry: 7
Description	Practiced searching and querying events in Splunk using its Search Processing Language (SPL), as a security analyst locating security events (such as failed logins and application errors) in a SIEM. This entry documents the use of a cybersecurity tool.
Tool(s) used	Splunk (Search & Reporting, SPL). I built searches from an index and a search term (<code>index=main fail</code>) and refined them with the core SPL techniques: pipng to transform results (<code>index=main fail chart count by host</code>, which counts events by host to surface machines with excessive failures), wildcards to broaden a term (<code>fail*</code> matches <code>failed</code>, <code>failure</code>, and similar), and double quotes to pin an exact phrase (<code>"login failure"</code>, rather than events that merely contain <code>login</code> or <code>failure</code> separately). In a raw log search against a fictional online store's data I ran <code>buttercupgames error OR fail*</code>, specifying the index, the Boolean <code>OR</code>, and the wildcard, then scoped it with the time-range picker (last 30 days), read the timeline and events viewer, and refined by excluding a data source with <code>host!=www1</code> so only the <code>www2</code> and <code>www3</code> hosts remained.
Findings from the searches	- Broad keyword searches (for example, <code>failed login</code>) can return thousands of results and slow the engine. Adding parameters such as an event ID and a date and time range narrows the search and speeds it up. - The Buttercup Games errors related to HTTP cookies on the website, and each event carried a timestamp, raw logged data, and source info (host, source, sourcetype). - A raw log search is slower because it extracts log fields at search time, and SPL provides commands to optimize performance.

Additional notes	The reusable idea is that specific queries beat broad ones (the index, exact terms, and a time range), and that piping, as in the Linux shell, chains commands to transform results into exactly the view you need, such as a per-host failure count. <i>Search methods practiced in the course's Splunk Cloud environment. The example queries use the course's demonstration datasets.</i>
-------------------------	--

Date: 22 July 2026	Entry: 8
Description	Practiced searching and querying events in Google Security Operations (Chronicle), locating events such as failed logins with a Unified Data Model (UDM) search, the SIEM counterpart to the Splunk work in Entry 7. This entry documents the use of a cybersecurity tool.
Tool(s) used	Google SecOps (Chronicle): UDM Search, with awareness of Raw Log Search and the YARA-L rule language. Using the structured query builder I ran a UDM search: <code>metadata.event_type = "USER_LOGIN" AND security_result.action = "BLOCK"</code> . Here, <code>metadata.event_type</code> selects authentication (user-login) events, the <code>AND</code> operator requires both conditions, and <code>security_result.action = "BLOCK"</code> narrows to blocked or failed logins. I read the results (the UDM search terms, a bar-graph timeline of failed logins over time for spotting patterns, and a list of timestamped events each tied to an asset or device), then opened an event's raw log and used Quick Filters and procedural filtering (for example <code>target.ip</code>) to narrow to a specific IP address.
Findings and reference notes	• UDM Search is the default and searches normalized data (indexed and structured, so faster). Raw Log Search searches un-normalized logs and is used to find non-normalized fields or troubleshoot ingestion, and it supports regular expressions. • Searchable fields include hostname, domain, IP, URL, email, username, and file hash, and the common UDM field groups are Entities, Event metadata, Network metadata, and Security results. • One result was a blocked login for a user named "alice." Chronicle uses YARA-L to define detection rules over ingested logs (for example, detecting data exfiltration).
Additional notes	Together with Entry 7, this shows the query concept transfers across SIEMs: you specify the data (the index or event type), the terms, boolean logic, and a time range, while each tool has its own language (SPL versus UDM and YARA-L) and its own trade-off between normalized and raw search. <i>Search methods practiced in the course's Google SecOps environment. The example query uses the course's demonstration data.</i>

Date: 23 July 2026	Entry: 9 (reflection)
------------------------------	---------------------------------

Description	A closing reflection on my learning journey through the incident detection and response work in this course, written as the final entry of the journal.
Reflections and notes	1. Were there any specific activities that were challenging for you? Why or why not? The packet analysis work with Wireshark and tcpdump was the most challenging. Coming from software engineering I was comfortable reading application logs, but a raw capture is a different skill: I had to learn to follow each request to its response across hosts and build layered filters, by IP, protocol, and port, to isolate the traffic that mattered. 2. Has your understanding of incident detection and response changed since taking this course? Somewhat. My engineering background already gave me strong instincts for tracing a problem to its root cause, so the biggest change was structural rather than conceptual. Frameworks like the NIST incident lifecycle, the 5 W's, and the IDS and SIEM workflow gave those instincts a repeatable shape and a shared vocabulary I can carry into any investigation. 3. Was there a specific tool or concept that you enjoyed the most? Why? The Pyramid of Pain and the 5 W's framework. I enjoyed the Pyramid of Pain because it reframes detection as economics: the higher up you detect, the more costly the attack becomes to sustain, which makes behaviour-based defence the goal. I liked the 5 W's for the opposite reason: its simplicity keeps an incident record complete and consistent every time.

Source materials & attribution

This is the complete, running incident handler's journal for my Google Cybersecurity Certificate work. A frozen two-entry snapshot, produced as the deliverable for the phishing incident response lab task, is preserved as `incident-handlers-journal.pdf` in that lab. The scenarios, lab environments, sample capture, alert ticket, and journal template are adapted from the Google Cybersecurity Certificate on Coursera:

- **Module 3 (Connect and Protect, Networks and Network Security):** the Wireshark "Analyze your first packet" and tcpdump "Capture your first packet" lab activities (Entries 3 and 4), and the related network traffic analysis reports.
- **Module 6 (Sound the Alarm, Detection and Response):** the ransomware and phishing scenarios, the VirusTotal file-hash and IoC analysis, the Suricata signatures-and-logs lab, and the Splunk and Chronicle SIEM search methods (Entries 1, 2, 5, 6, 7, and 8).

The packet captures and filtering (Entries 3 and 4) and the Suricata rule and log analysis (Entry 6) were performed in the course's Linux and Windows lab machines. Those environments and the sample files (`sample.pcap`, `custom.rules`) are course-provided and remained in the lab, so those entries record my own workflow, commands, and findings rather than redistributing the lab files. The Splunk and Chronicle searches (Entries 7 and 8) were practiced in the course's cloud SIEM environments, and the example queries use the course's demonstration datasets. The 5 W's write-ups, the playbook-driven escalation decision, the VirusTotal IoC analysis, the reflection, and all entries in this journal are my own work.